

PGM coursework 2 part (i)

02300963

Variational Autoencoder trained on MNIST

Task 1: Baseline VAE and training

1. Model setup

A baseline variational autoencoder (VAE) with fully connected layers was trained on MNIST. Each 28×28 image was flattened into a 784-dimensional vector. The encoder used two hidden layers of sizes 512 and 256, and mapped the input to a 20-dimensional latent space. The decoder mirrored this structure and reconstructed the image through a sigmoid output layer.

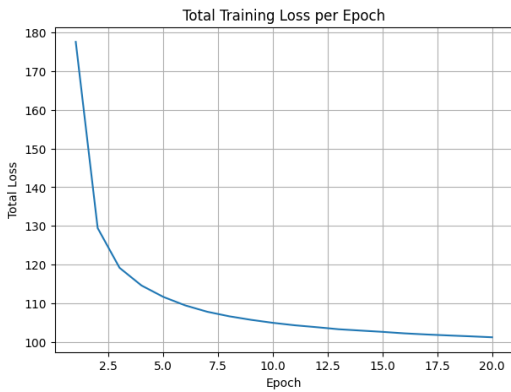
The latent dimension was set to 20 as a compromise between representation capacity and regularisation: it is large enough to capture digit variability, while remaining sufficiently constrained for stable generation.

2. Training objective

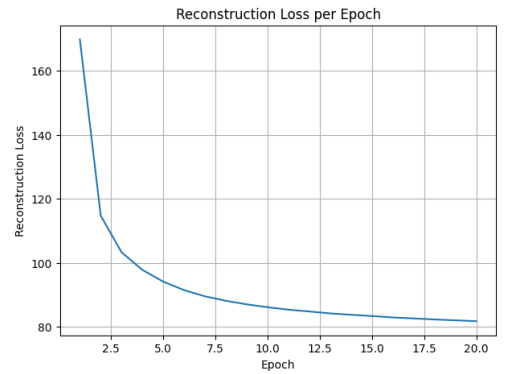
The model was trained using the standard VAE objective, consisting of a reconstruction term and a KL regularisation term. The reconstruction term encourages accurate recovery of the input image, while the KL term keeps the latent distribution close to the Gaussian prior. This balance is necessary for obtaining both good reconstructions and meaningful samples from the prior.

3. Training behaviour

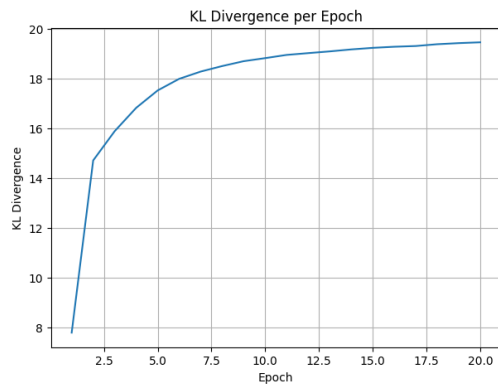
Figure 1 shows the training curves for the total loss, reconstruction loss, and KL divergence.



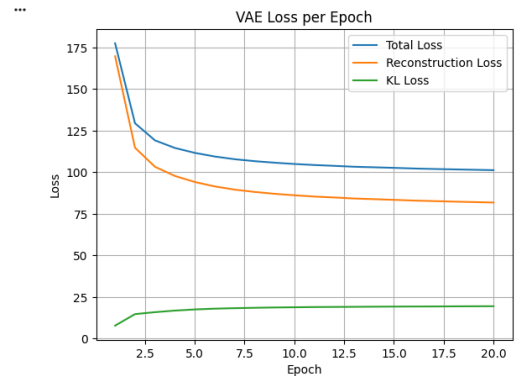
(a) Total training loss against epoch.



(b) Reconstruction loss against epoch.



(c) KL divergence against epoch.



(d) Comparison of total, reconstruction, and KL losses.

Figure 1: Training curves of the baseline VAE on MNIST.

The total and reconstruction losses decrease rapidly in the early epochs, showing that the model quickly learns the main digit structure. The KL term rises initially and then stabilises, indicating that the latent representation becomes progressively organised. After about 10 epochs, all curves flatten noticeably, so further improvement is limited.

4. Choice of training epochs

The model was trained for 20 epochs. This choice was based on the loss curves: most improvement occurs early in training, while after roughly 10 epochs the gains become small. Thus, 20 epochs provide a reasonable balance between convergence and computational cost.

Task 2: Reconstruction vs generation

1. Comparison of Generated Samples

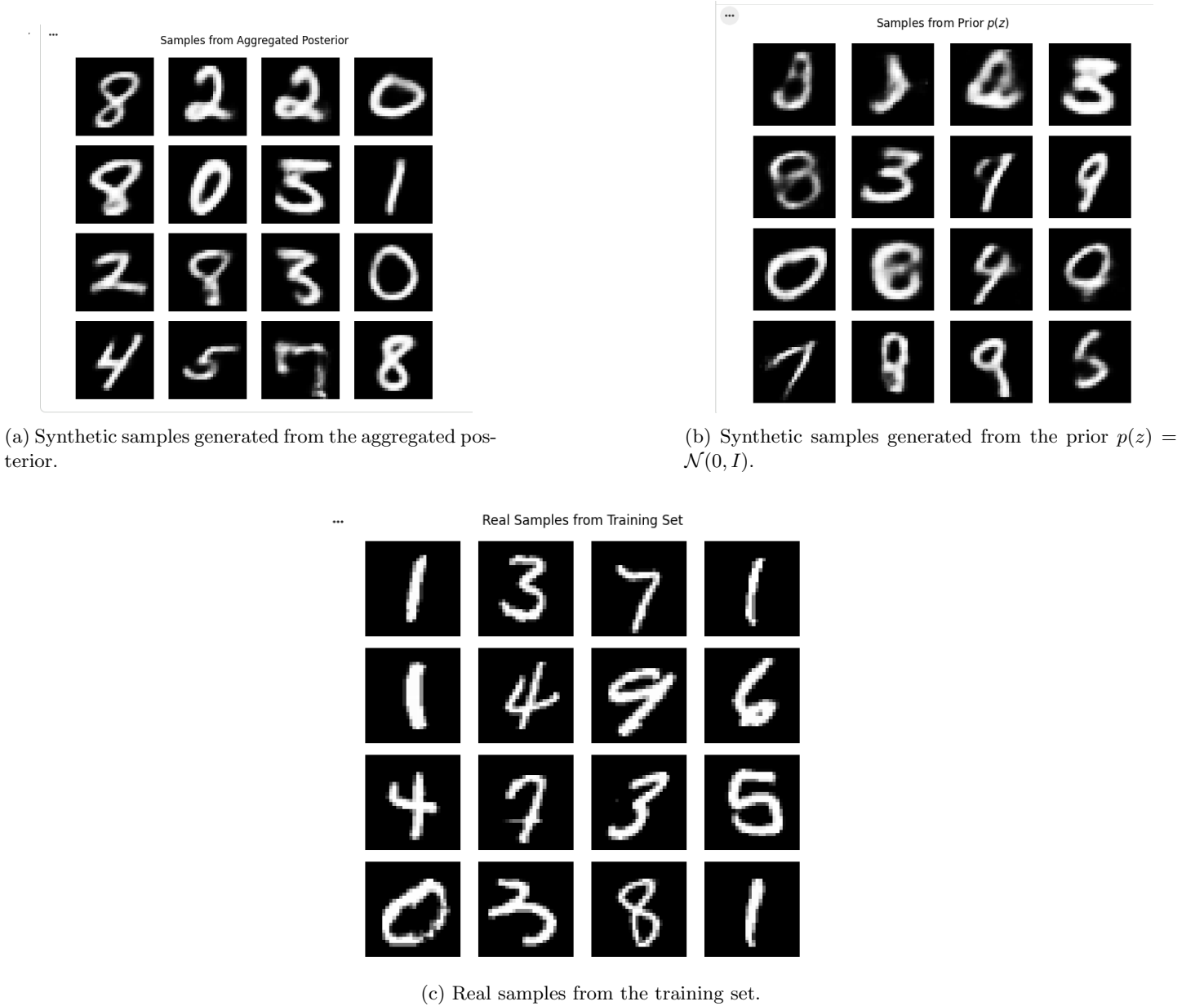


Figure 2: Comparison of real MNIST samples and images generated by the trained VAE. The first panel shows samples drawn from the aggregated posterior, the second shows samples drawn from the prior distribution, and the third shows the real training samples.

As shown in Figure 2, samples drawn from the aggregated posterior are generally clearer than those drawn directly from the prior.

2. Discussion of Prior and Aggregated Posterior Sampling

The real training samples provide a reference for the visual quality of genuine MNIST digits. Samples generated from the aggregated posterior are generally clearer and more digit-like than those generated directly from the prior. This is because the aggregated posterior is formed from latent codes inferred from real data, so these samples lie in regions of latent space that are well supported during training.

In contrast, prior samples are drawn directly from $p(z) = \mathcal{N}(0, I)$. Although the KL divergence term encourages the approximate posterior $q(z|x)$ to be close to the prior, the match is not perfect in practice. As a result, some latent points sampled from the prior may fall into regions that are less frequently visited by the encoder during training. The decoder is therefore less able to map such points to high-quality digit images, which explains why prior generation may produce blurrier or lower-quality samples even when reconstruction is accurate.

Task 3: Comparing the prior and the aggregated posterior

1. PCA comparison

To compare the latent prior with the aggregated posterior, both sets of latent samples were projected onto the first two principal components.

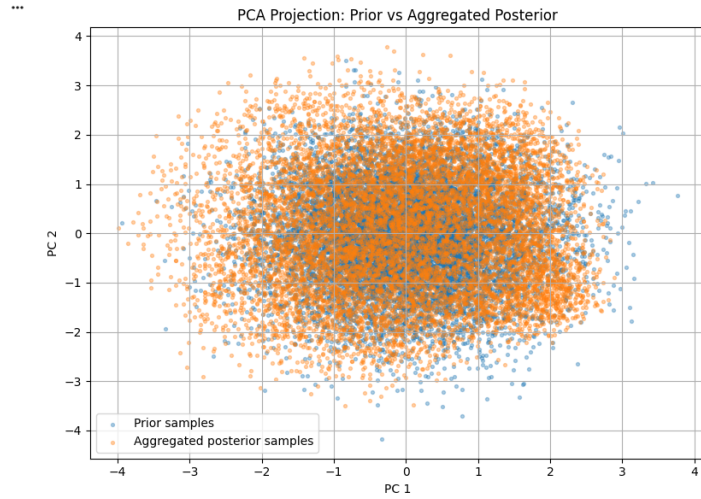


Figure 3: PCA projection of latent samples from the prior and the aggregated posterior.

Figure 3 shows substantial overlap between the two distributions in the PCA plane, suggesting that the aggregated posterior is broadly close to the standard Gaussian prior. However, the match is not exact: the posterior samples appear slightly shifted and more dispersed. This indicates that the KL regularisation brings the latent distribution towards the prior, but does not eliminate the mismatch completely.

To examine latent structure further, the aggregated posterior samples were also coloured by digit label.

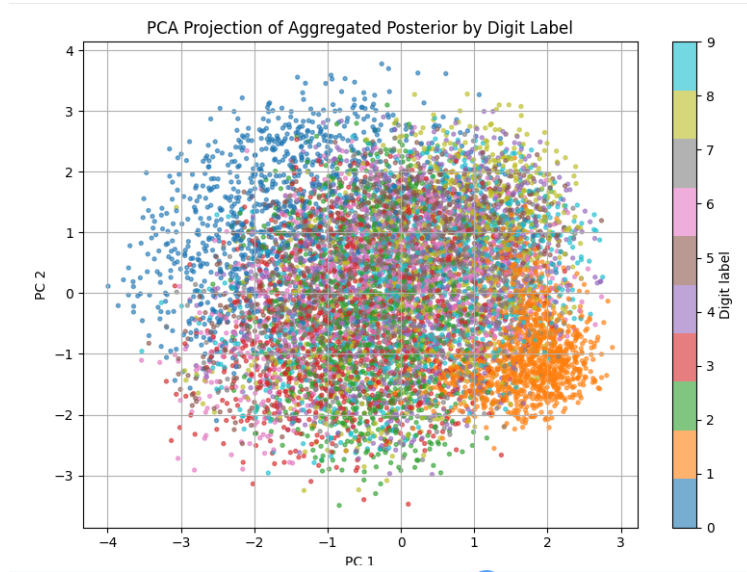


Figure 4: PCA projection of aggregated posterior samples coloured by digit label.

Figure 4 shows that samples with the same label tend to cluster together, although different digits still overlap substantially. This suggests that the latent space retains meaningful data-dependent structure rather than behaving as a purely isotropic Gaussian cloud.

2. Wasserstein distance analysis

To quantify the discrepancy, the aggregated posterior was approximated by a Gaussian distribution using its empirical mean and covariance, and the 2-Wasserstein distance to the prior was computed. The observed distance was

$$W_2 \approx 0.303.$$

To interpret the observed Wasserstein distance, a same-distribution baseline was constructed. Specifically, two independent samples of equal size were repeatedly drawn from the same distribution, a Gaussian was fitted to each sample, and the Wasserstein distance between the two fitted Gaussians was computed. These repeated comparisons provide a reference scale for the distance expected from finite-sample variation alone.

The baseline distances are centred around 0.16, while the observed prior-versus-posterior distance is 0.303. Because this value is substantially larger and lies above the 95% baseline quantile, the mismatch is unlikely to be due to sampling noise alone.

To assess whether this discrepancy is substantial, a same-distribution baseline was constructed by repeatedly comparing Gaussian fits from two independent samples drawn from the same distribution. The baseline results were:

```

1 Baseline prior vs prior:
2   mean = 0.1578, std = 0.0073
3   95% quantile = 0.1692
4 Baseline fitted vs fitted:
5   mean = 0.1597, std = 0.0063
6   95% quantile = 0.1688
7 Observed distance = 0.3030

```

The observed distance is roughly twice the baseline mean and well above the 95% quantile, so the mismatch is larger than can be explained by sampling noise alone.

This is consistent with the results of Task 2. Since the decoder is mainly trained on latent codes coming from the aggregated posterior, samples generated from that region are naturally clearer than samples drawn directly from the idealised prior. The Wasserstein analysis therefore provides quantitative support for the visual difference observed earlier.

Task 4: Reconstruction and regularisation

Theoretical effect of changing β In a β -VAE, the parameter β controls the trade-off between accurate reconstruction and closeness of the latent distribution to the prior. When β is small, the reconstruction term dominates the objective, so the encoder is encouraged to retain more input-specific information in the latent code. This usually improves reconstruction quality, but allows the latent distribution to deviate from the standard Gaussian prior.

As β increases, the KL term is weighted more heavily, so the latent distribution is pushed closer to $\mathcal{N}(0, I)$. This improves regularisation and makes sampling from the prior more meaningful, but it also restricts how much information can be encoded in the latent variable. As a result, reconstruction quality typically worsens. If β becomes too large, the model may enter posterior collapse, where deviations from the prior are penalised so strongly that the latent code becomes almost uninformative and the decoder produces vague average-like outputs.

Empirical effect on reconstruction quality The reconstruction results confirm this trade-off. For very small β , such as $\beta = 0.01$, the reconstructed digits remain sharp and visually close to the inputs, showing that the model preserves detailed information in the latent code. As β increases to moderate values such as $\beta = 0.5$ and $\beta = 5$, the reconstructions become progressively blurrier. When $\beta = 100$, the reconstructed images are often no longer clearly recognisable as digits, indicating severe loss of input information.

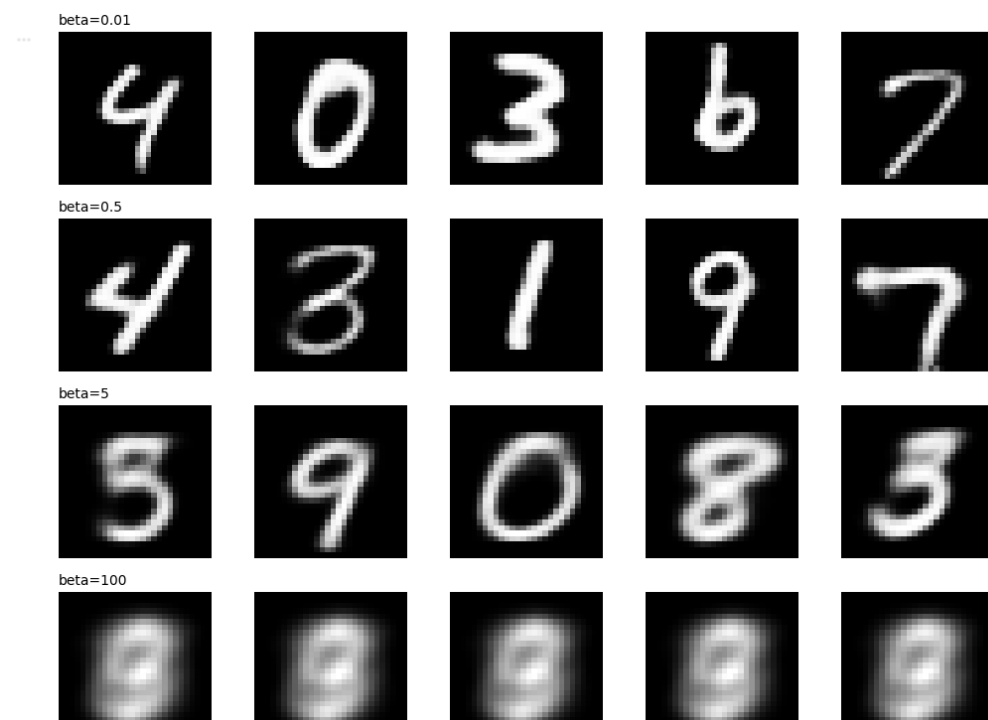


Figure 5: Reconstruction results for different values of β .

Empirical effect on the structure of the latent space The latent space becomes increasingly regularised as β grows. Figure 6 shows that the KL-related quantity changes systematically with β , while Figure 7 shows that the Wasserstein distance between the aggregated posterior and the prior decreases as β increases. This indicates that larger β pushes the latent distribution closer to $\mathcal{N}(0, I)$, producing a more constrained and regular latent space.

However, this does not necessarily imply a better representation. For small β , the latent space is freer to preserve digit-dependent structure, which supports accurate reconstruction but leads to a larger mismatch with the prior. For very large β , the latent space becomes over-regularised: although it is closer to the prior, it loses much of its input-specific organisation. Therefore, increasing β improves proximity to the prior, but excessive regularisation reduces the usefulness of the latent representation.

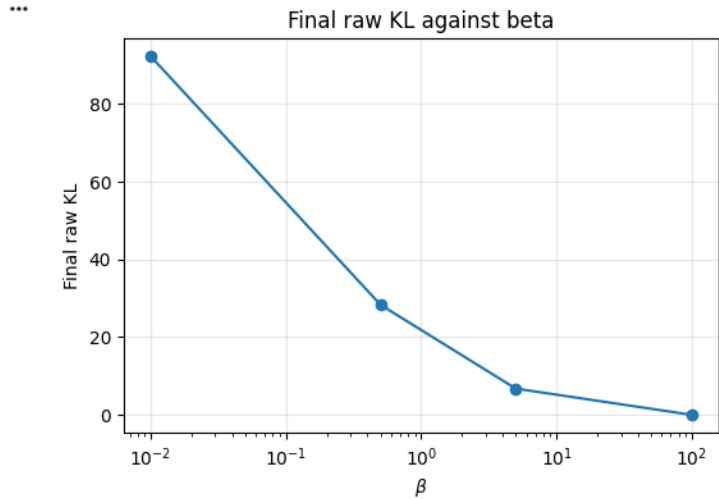


Figure 6: Final KL-related quantity as a function of β .

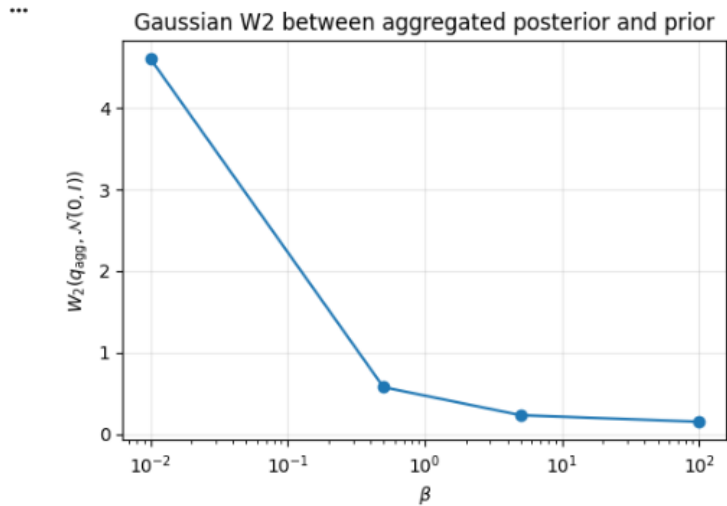


Figure 7: W_2 distance between the aggregated posterior and the prior for different values of β .

Empirical effect on generated samples The effect of β on generation is different from its effect on reconstruction. When latent codes are sampled directly from the prior, the model with very small β performs poorly: although reconstruction is strong, many generated digits are blurry or distorted. This suggests that the latent distribution learned by the encoder is not well aligned with the prior, so prior samples often fall outside the regions where the decoder was effectively trained.

As β increases to moderate values, generated samples become more coherent and recognisable, indicating that improved alignment with the prior leads to better generation. However, when β is too large, generation quality deteriorates again. At $\beta = 100$, the outputs collapse to very similar blurred images, which is consistent with posterior collapse and an almost uninformative latent code.

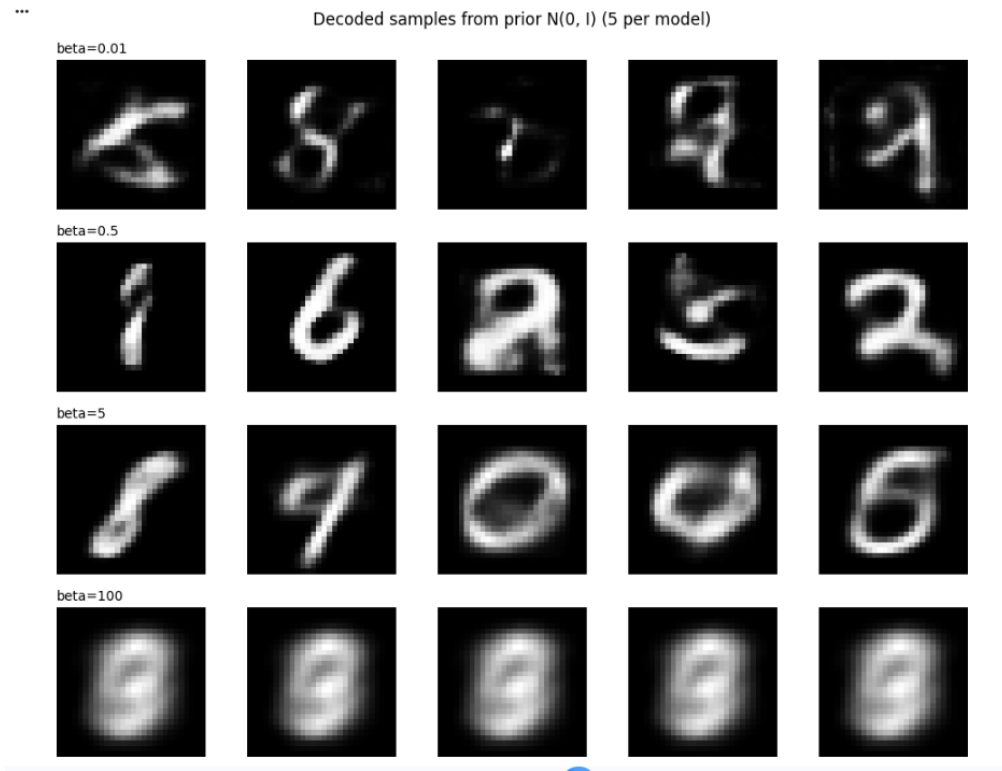


Figure 8: Samples generated from the prior for different values of β .

Can an appropriate choice of β improve generation? The experiments suggest that an appropriate choice of β can improve generation compared with the standard VAE objective. A small β gives the best reconstruction quality, but prior generation remains weak because the latent distribution is not sufficiently close to the prior. Increasing β improves this alignment and makes prior sampling more meaningful. In the experiments, moderate values such as $\beta \approx 5$ give more coherent generated digits than the standard VAE case.

However, this improvement only holds within a moderate range. If β becomes too large, the KL term dominates the objective and prevents the model from encoding useful input information, so both reconstruction and generation collapse. Therefore, β should be chosen as a compromise: neither too small, which gives poor prior generation, nor too large, which leads to over-regularisation. In these experiments, the most suitable range appears to be approximately $\beta \in [4, 6]$.

Task 5: Improved architecture

1. Choice of K

To select a suitable number of convolutional layers, we compared architectures with $K = 1$ and $K = 2$. Increasing K from 1 to 2 reduces the total training loss from about 121 to 102, mainly through a lower reconstruction loss, while the KL term remains broadly similar. This suggests that adding a second convolutional layer improves the model's ability to capture the spatial structure of MNIST digits.

The improvement, however, comes with a higher computational cost. In our experiments, the $K = 1$ model required about 7 minutes of training, while the $K = 2$ model required about 13 minutes. Although deeper architectures might reduce the loss further, the gain is likely to become small relative to the additional cost. For this reason, $K = 2$ was chosen as a reasonable balance between model complexity and efficiency.

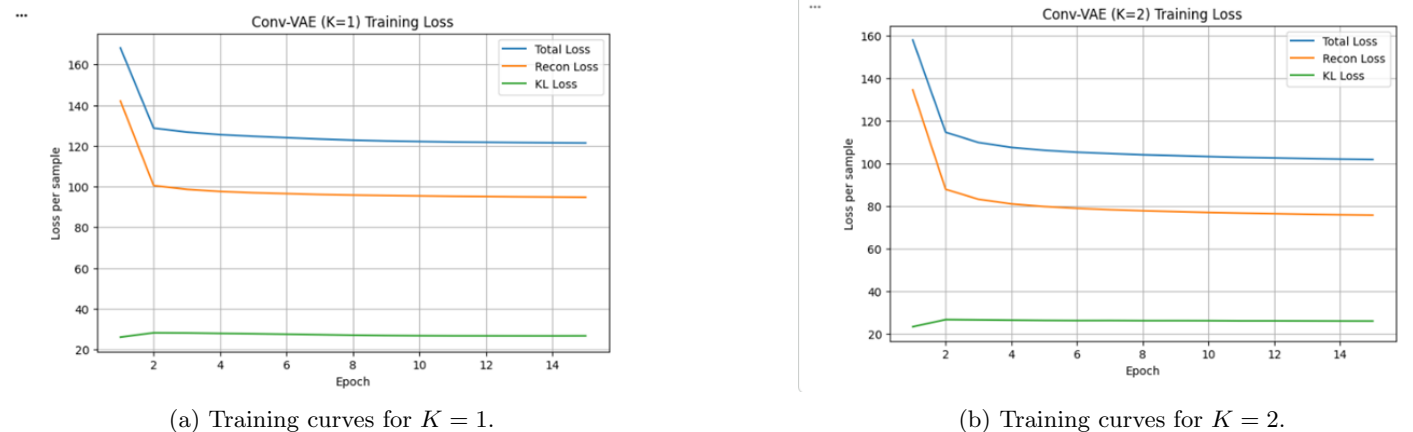


Figure 9: Training loss curves for convolutional VAEs with different numbers of convolutional layers.

2. Reconsidering Tasks 2 and 3 for the convolutional VAE

We now reconsider the analyses from Tasks 2 and 3 using the convolutional VAE with $K = 2$, focusing on sample quality and the relationship between the aggregated posterior and the prior.

Samples from the aggregated posterior. Figure 10 compares samples generated from the aggregated posterior for the baseline fully-connected VAE and the convolutional VAE. The two models give broadly similar results, but the convolutional model produces slightly cleaner stroke patterns and reduces some local blurring. This suggests that the convolutional architecture captures local image structure more effectively, which mainly benefits reconstruction-related behaviour.

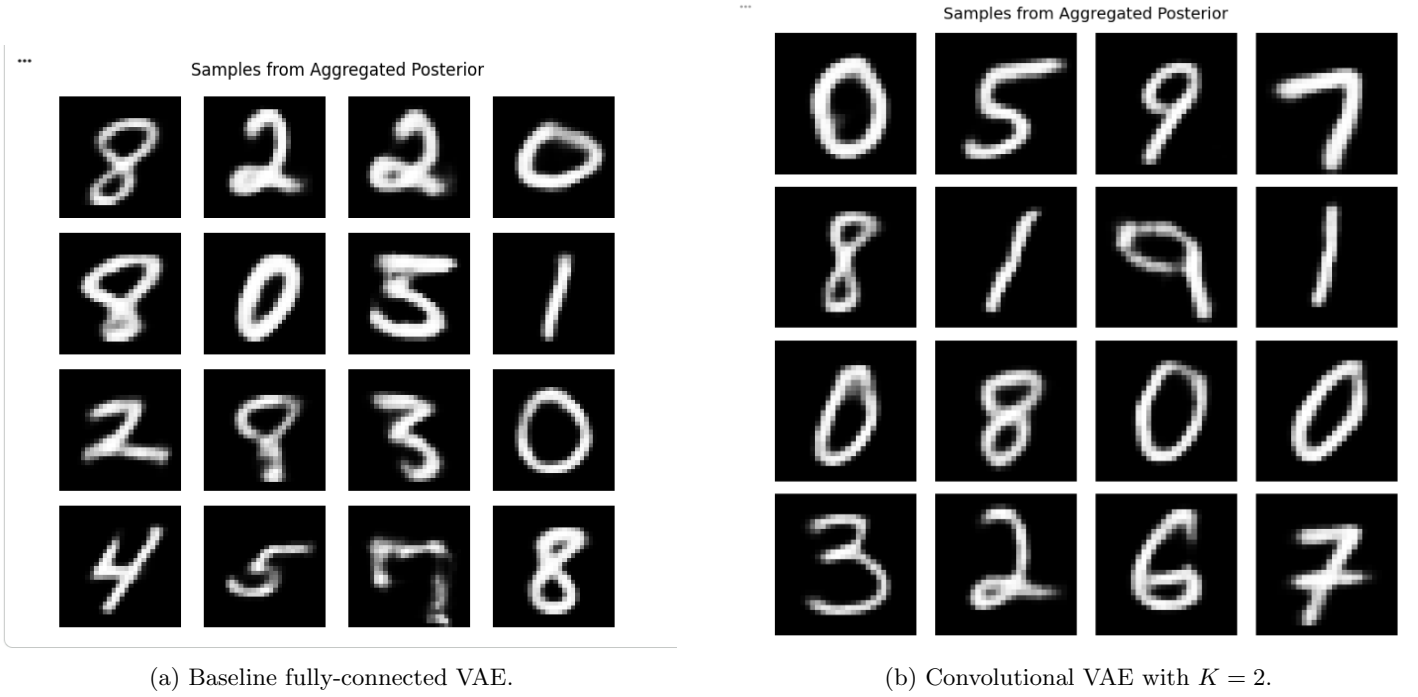


Figure 10: Comparison of samples generated from the aggregated posterior.

Samples from the prior. The comparison is different when latent codes are sampled directly from the prior. As shown in Figure 11, prior samples remain weak in both models, and the convolutional VAE does not improve this behaviour. In fact, its prior samples appear slightly less interpretable in some cases. This shows that improving the image modelling architecture does not by itself guarantee better generation from the assumed prior.

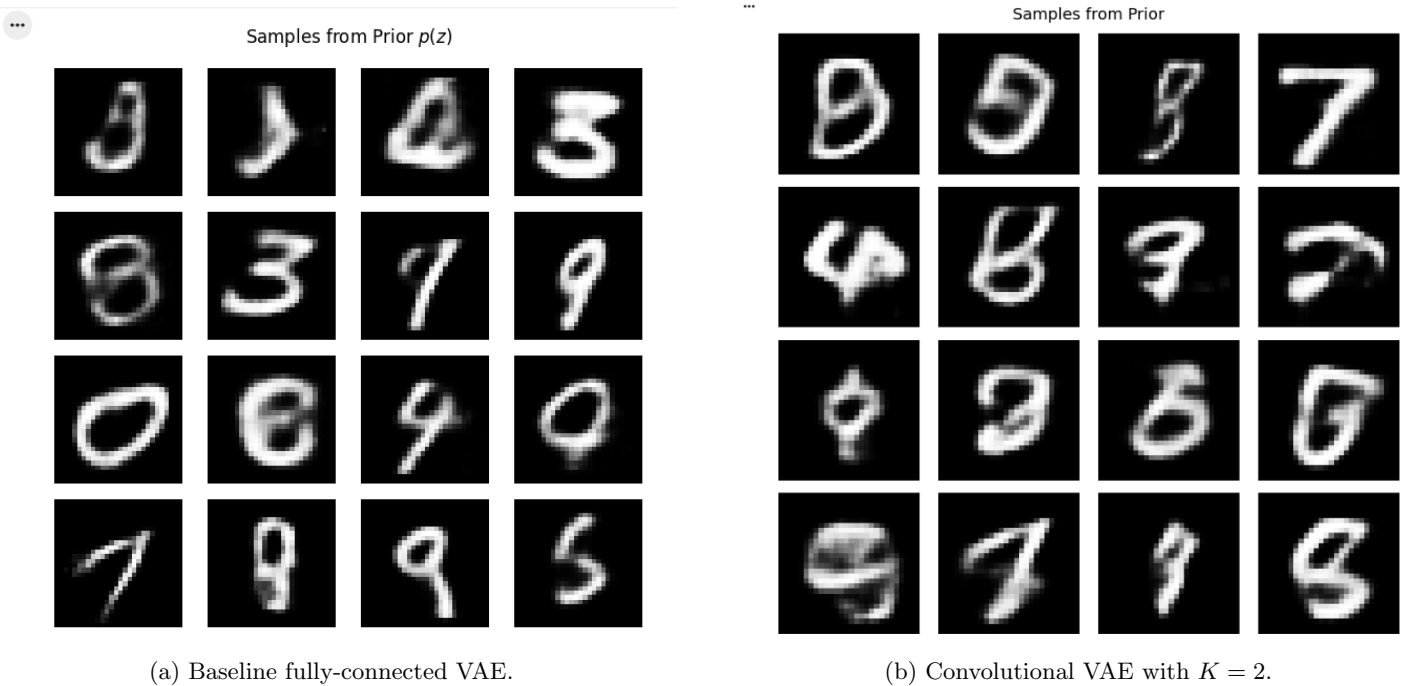


Figure 11: Comparison of samples generated from the prior.

Latent-space structure. To analyse the latent space, we compare the aggregated posterior with the Gaussian prior using PCA. Figure 12 shows that the aggregated posterior of the convolutional VAE does not overlap closely with the prior and

appears more clearly separated than in the baseline model. Figure 13 further shows visible clustering by digit label, indicating that the latent representation preserves meaningful class-dependent structure.

This structure is useful for reconstruction, but it also suggests that the aggregated posterior is concentrated in specific regions of latent space rather than matching the standard Gaussian prior closely.

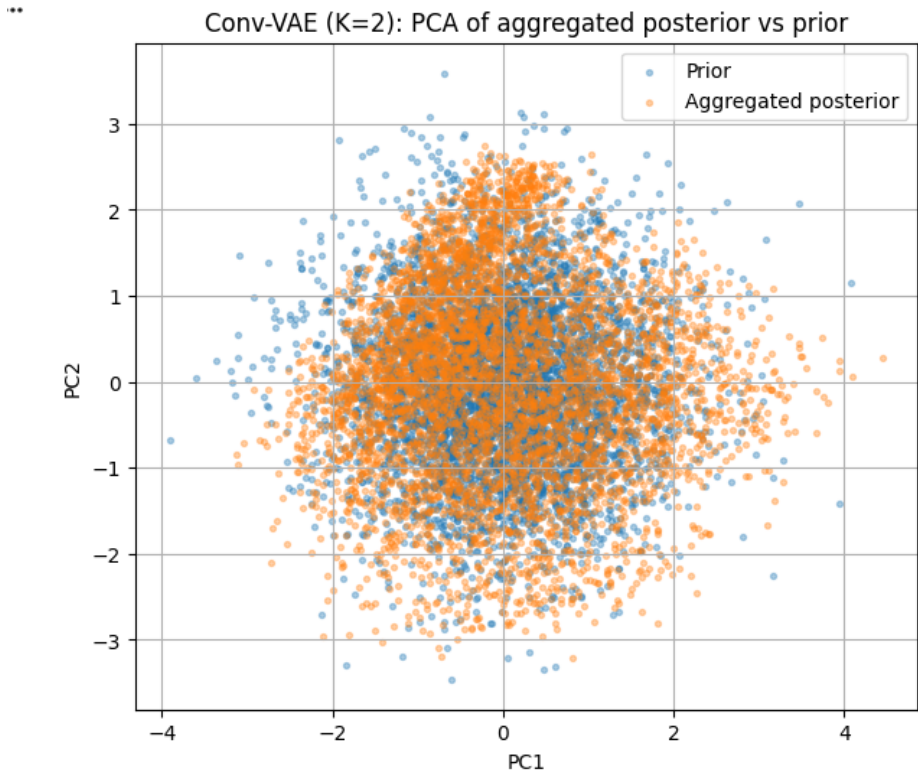


Figure 12: PCA projection of latent samples from the prior and the aggregated posterior for the convolutional VAE with $K = 2$.

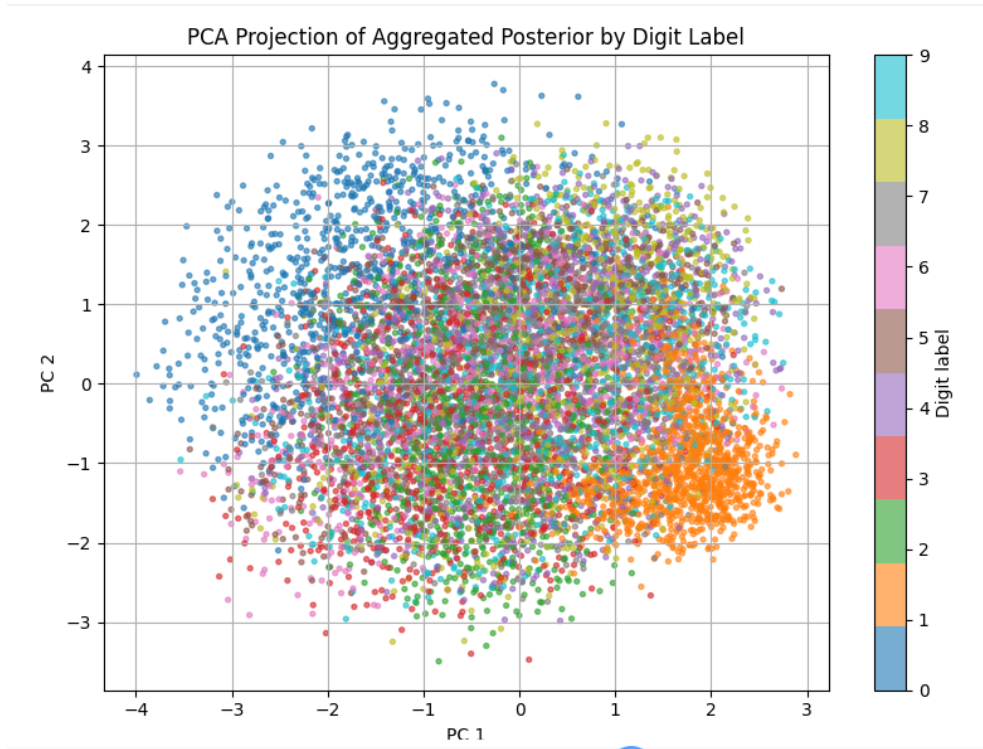


Figure 13: PCA projection of the aggregated posterior for the convolutional VAE with $K = 2$, coloured by digit label.

3. Quantitative and qualitative comparison with the baseline

The convolutional architecture improves the model quantitatively in terms of training loss, mainly by reducing the reconstruction component. This is consistent with the visual comparison of posterior-based samples, where the convolutional model produces slightly cleaner digits.

However, the relationship between the aggregated posterior and the prior becomes worse. Using a Gaussian approximation, the Wasserstein distance between the aggregated posterior and the prior is

```
1 Baseline fully-connected VAE:
2 W2 = 0.3030 | mean_part = 0.07635 | cov_part = 0.0647
3
4 Convolutional VAE with K=2:
5 W2 = 0.4936 | mean_part = 0.0371 | cov_part = 0.2065
```

The larger W_2 value for the convolutional model shows that its aggregated posterior is further from the assumed prior than in the baseline model. This quantitative result agrees with the PCA plots, where the mismatch is visually more pronounced.

Therefore, the convolutional VAE performs better in reconstruction-related behaviour, but worse in terms of alignment between the learned latent distribution and the Gaussian prior. This also explains why prior-based generation does not improve, despite the stronger image modelling capacity.

4. Discussion

Overall, the convolutional architecture improves performance only partially. It gives lower training loss and slightly better posterior-based sample quality because convolutional layers are better at capturing local spatial patterns such as edges and stroke continuity. This makes the model more effective at reconstructing structured image data such as MNIST digits.

However, the improved decoder and encoder flexibility also allow the model to organise the latent space in a more data-dependent way, which increases the mismatch between the aggregated posterior and the standard Gaussian prior. As a result, direct generation from the prior does not improve and may even become slightly worse.

Therefore, the new architecture improves reconstruction and posterior-supported generation, but it does not improve overall generative performance from the prior. Its main advantage is better image modelling, while its main limitation is weaker alignment between the learned latent distribution and the assumed prior.

MATH70148 - Probabilistic Generative Models 2025-2026

Coursework 2 - Part II

Instructions:

You will implement a minimal version of the GPT architecture and train it to generate synthetic text in the style of the Shakespeare's works. Follow this notebook and complete the missing parts according to the given instructions. Register the code you generate in the parts indicated as (YOUR CODE HERE) and your responses (when applicable) by creating a new cell in this notebook. Your submission will be assessed from this notebook only, **no pdf report is needed**.

Comment on running time: It is expected that this model is to be trained over at least 5000 epochs to produce text resembling the structure of a play, and you are strongly encouraged to run it on Google Colab (where the 5000 epochs should take around 6 minutes if using a T4 runtime type) or a computer with a sufficiently capable GPU. For reference, if ran on a regular MPS-enabled Apple M3 Pro Macbook Pro, this code will take ~10 minutes, and ~5.5 minutes if using a NVIDIA GeForce GTX 1080 GPU.

```
import torch
import torch.nn as nn
from torch.nn import functional as F
```

1. Download the corpus and tokenise it

```
# If using Colab, consider:
#from google.colab import drive
#drive.mount('/content/drive/')
```

```
# Load the provided dataset
```

```
# Read the entire Shakespeare corpus into memory
# We explicitly set UTF-8 encoding to avoid platform-dependent issues.
with open('shakespeare.txt', 'r', encoding='utf-8') as file:
    text = file.read()

# Basic corpus statistics
num_chars = len(text)

print("=== Corpus Statistics ===")
print(f"Total number of characters: {num_chars:,}")
print(f"First 200 characters preview:\n")
print(text[:200])
```

```
=== Corpus Statistics ===
Total number of characters: 1,115,393
First 200 characters preview:
```

```
First Citizen:
Before we proceed any further, hear me speak.
```

```
All:
Speak, speak.
```

```
First Citizen:
You are all resolved rather to die than to famish?
```

```
All:
Resolved. resolved.
```

```
First Citizen:
First, you
```

```
# Extract the set of unique characters appearing in the text
# `set(text)` removes duplicates
# `sorted(...)` ensures a deterministic ordering (important for reproducibility)
chars = sorted(list(set(text)))
```

```
# Size of the vocabulary (number of unique characters)
vocab_size = len(chars)
```

```
# Display results
print("Unique characters (sorted):")
print("".join(chars)) # Join list into a readable string
```

```
print("\nVocabulary size:")
print(f"{vocab_size} unique characters")
```

Unique characters (sorted):

```
!$%&',-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
```

Vocabulary size:
65 unique characters

```
# -----
# Character-level tokenizer
# -----

# NOTE:
# The model will operate at the character level, meaning that the tokens are characters (and not words)

# Create a mapping from character to integer index
# This is our "string-to-index" (stoi) dictionary.
stoi = {char: idx for idx, char in enumerate(chars)}

# Create the inverse mapping: index to character
# This is needed to convert model outputs back to text.
itos = {idx: char for idx, char in enumerate(chars)}

def encode(s):
    """
    Convert a string into a list of integer token IDs.

    Args:
        s (str): Input string.

    Returns:
        List[int]: List of corresponding character indices.
    """
    return [stoi[char] for char in s]

def decode(indices):
    """
    Convert a list of integer token IDs back into a string.

    Args:
        indices (List[int]): List of character indices.

    Returns:
        str: Decoded string.
    """
    return ''.join(itos[idx] for idx in indices)
```

```
# Convert the entire corpus into a PyTorch tensor of token IDs.
# We use dtype=torch.long because embedding layers expect integer (int64) indices.
data = torch.tensor(encode(text), dtype=torch.long)

# Inspect the shape of the resulting tensor
print("Data tensor shape:", data.shape)
```

Data tensor shape: torch.Size([1115393])

```
# Display a short excerpt of the original text and its corresponding
# integer (token ID) representation to illustrate the encoding process.

N = 50

print(f"Text containing the first {N} characters:")
print("-" * 50)
print(text[:N])

print("\n" + "-" * 50)
print("Its representation as a PyTorch tensor:\n")
print(data[:N])
```

Text containing the first 50 characters:

```
-----
First Citizen:
Before we proceed any further, hear
```

```
-----
Its representation as a PyTorch tensor:
```

```
tensor([[18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 14, 43, 44,
        53, 56, 43, 1, 61, 43, 1, 54, 56, 53, 41, 43, 43, 42, 1, 39, 52, 63,
        1, 44, 59, 56, 58, 46, 43, 56, 6, 1, 46, 43, 39, 56]])
```

2. Dataset split

```
# Split the dataset into training (90%) and validation (10%) sets.
# We consider a sequential split to preserve temporal structure.

n = int(0.9 * len(data)) # 90% for training
train_data = data[:n]
val_data = data[n:]

print(f"Training corpus size: {len(train_data):,} characters "
      f"({train_data.shape[0] / data.shape[0]:.2%} of total)")

print(f"Validation corpus size: {len(val_data):,} characters "
      f"({val_data.shape[0] / data.shape[0]:.2%} of total)")
```

```
Training corpus size: 1,003,853 characters (90.00% of total)
Validation corpus size: 111,540 characters (10.00% of total)
```

On the data-label mechanics

- We access the data through *context windows*: contiguous blocks of text in numerical (token) representation of size `block_size`.
- The model is self-supervised. That is, we train it so that, given a sequence of tokens $x_{i:j}$, it predicts the next token x_{j+1} .
- The labels are obtained from the same context window by shifting it with an offset of 1 position.

```
# EXAMPLE:

# Given a context window of size 7,
# we construct an input-target pair (X, Y),
# where Y is simply X shifted by one position.

block_size = 7

print("A context window (X, Y) looks like:\n")

# Input sequence (context)
X = data[:block_size]
print(f"X (token IDs): {[x.item() for x in X]}")
print(f"--> decode(X): {decode([x.item() for x in X])}")

print("-" * 40)

# Target sequence (shifted by one position)
Y = data[1:block_size + 1]
print(f"Y (token IDs): {[y.item() for y in Y]}")
print(f"--> decode(Y): {decode([y.item() for y in Y])}")
```

A context window (X, Y) looks like:

```
X (token IDs): [18, 47, 56, 57, 58, 1, 15]
--> decode(X): First C

-----
Y (token IDs): [47, 56, 57, 58, 1, 15, 47]
--> decode(Y): first Ci
```

```
# EXAMPLE (continued):

# However, within each context window we use the information
# in an autoregressive manner, generating multiple training
# examples from a single block.

x = train_data[:block_size]
y = train_data[1:block_size + 1]

for t in range(block_size):
    context = x[:t + 1] # Tokens available at time step t
    target = y[t]       # Next token to predict

    print(f"When the input is {context.tolist()}, the target is: {target.item()}")
```

```
When the input is [18], the target is: 47
When the input is [18, 47], the target is: 56
When the input is [18, 47, 56], the target is: 57
When the input is [18, 47, 56, 57], the target is: 58
```

```

When the input is [18, 47, 56, 57, 58], the target is: 1
When the input is [18, 47, 56, 57, 58, 1], the target is: 15
When the input is [18, 47, 56, 57, 58, 1, 15], the target is: 47

```

Therefore, each context window generates a number of training observations equal to its length.

We now turn to *batches*, which allows us to process multiple context windows in parallel. The key point is that each context window is independent: there is no computation that occurs across windows, only parallel computation within them. In other words, the autoregressive sequences corresponding to different context windows are not mixed. See the following example.

```

torch.manual_seed(1337)

batch_size = 4
block_size = 8 # maximum context length considered for prediction
# These hyperparameters will be redefined for the final training phase

def get_batch(split):
    """
    Generate a small batch of input (x) and target (y) sequences.

    Each batch contains `batch_size` independent context windows
    of length `block_size`, sampled randomly from the dataset.
    """
    data_split = train_data if split == 'train' else val_data

    # Random starting positions for each context window
    ix = torch.randint(len(data_split) - block_size, (batch_size,))

    # Construct input and target batches
    x = torch.stack([data_split[i:i + block_size] for i in ix])
    y = torch.stack([data_split[i + 1:i + block_size + 1] for i in ix])

    return x, y

# Sample a training batch
xb, yb = get_batch('train')

print("Inputs:")
print("Shape:", xb.shape) # (batch_size, block_size)
print(xb)

print("\nTargets:")
print("Shape:", yb.shape) # (batch_size, block_size)
print(yb)

```

```

Inputs:
Shape: torch.Size([4, 8])
tensor([[53, 59,  6,  1, 58, 56, 47, 40],
        [49, 43, 43, 54,  1, 47, 58,  1],
        [13, 52, 45, 43, 50, 53,  8,  0],
        [ 1, 39,  1, 46, 53, 59, 57, 43]])

Targets:
Shape: torch.Size([4, 8])
tensor([[59,  6,  1, 58, 56, 47, 40, 59],
        [43, 43, 54,  1, 47, 58,  1, 58],
        [52, 45, 43, 50, 53,  8,  0, 26],
        [39,  1, 46, 53, 59, 57, 43,  0]])

```

Questions

- Q1 (5% marks): Explain why autoregressive language modeling is considered self-supervised. What are the inputs and labels in this setup?
- Q2 (5% marks): From a context window of size T , how many training examples are generated? Explain why.

Q1. Why is autoregressive language modelling self-supervised? What are the inputs and labels?

Autoregressive language modelling is considered *self-supervised* because the training signal is obtained directly from the raw corpus, without requiring manually annotated labels. The text itself provides both the input and the supervision target. More precisely, given a token sequence

$$(x_1, x_2, \dots, x_T),$$

the model is trained to predict the next token at each position, so the corresponding target sequence is

$$(x_2, x_3, \dots, x_{T+1}).$$

Hence, the labels are obtained automatically by shifting the original sequence by one position.

In this setup, the **inputs** are the observed tokens in the current context window, while the **labels** are the next tokens that follow them in the corpus. This makes the learning procedure self-supervised, since the supervision is generated from the structure of the data itself rather than from external human annotation.

Q2. From a context window of size T , how many training examples are generated? Explain why.

A context window of size T generates T training examples. This is because each position in the window defines a valid next-token prediction task. Concretely, the model first uses

$$x_1 \rightarrow x_2,$$

then

$$(x_1, x_2) \rightarrow x_3,$$

then

$$(x_1, x_2, x_3) \rightarrow x_4,$$

and continues in this autoregressive manner until

$$(x_1, x_2, \dots, x_T) \rightarrow x_{T+1}.$$

Therefore, every token position in the input window contributes one supervision signal, giving a total of T training examples.

Equivalently, for an input sequence of length T , the model produces T aligned predictions, one for each position, against the shifted target sequence

$$(x_2, x_3, \dots, x_{T+1}).$$

3. Self-attention

- Q3 (15% marks): Implement the missing parts in the attention head class. Recall that in this model you need to implement masked attention.

```
# Embedding dimension (model "width"): each token is represented as a vector in  $\mathbb{R}^{\{n\_embd\}}$ 
n_embd = 64
dropout = 0.0

class Head(nn.Module):
    """
    A single self-attention head (causal / autoregressive).

    Given an input sequence of token embeddings x of shape (B, T, n_embd),
    this module produces an output of shape (B, T, head_size) by:
    1) projecting x into queries, keys, and values
    2) computing attention weights via scaled dot-product attention
    3) applying a causal mask so tokens cannot attend to the future
    4) computing a weighted sum of value vectors
    """

    def __init__(self, head_size: int):
        super().__init__()

        # ----- #
        # -----[YOUR CODE HERE]----- #
        # ----- #

        # Causal mask (lower-triangular): prevents attending to future positions.
        # register_buffer makes it part of the module state (moves with .to(device))
        # but it is not a learnable parameter.
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False) #Obtain K,Q,V

        self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size))) #Use the up-triangle matrix, so th

        # Dropout applied to the attention weights (common regularization in Transformers)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:

        """
```



```

Args:
    x: (B, T, n_embd) input token embeddings

Returns:
    out: (B, T, head_size) attention output for this head
"""

# ----- #
# -----[YOUR CODE HERE]----- #
# ----- #
B, T, E = x.shape #Batch size/ Sequence length/ embedding dimension(Not been used)
k = self.key(x)    # (B, T, head_size) to obtain the key
q = self.query(x)  # (B, T, head_size) to obtain the query

wei = q @ k.transpose(-2, -1) * (k.shape[-1]** -0.5) # (k.shape[-1]** -0.5) is the part 1/sqrt(c)
wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # Apply casual mask.
wei = F.softmax(wei, dim=-1) #Using softmax to get a probability vector
wei = self.dropout(wei)

v = self.value(x) # (B, T, head_size)
out = wei @ v      # (B, T, head_size)

return out

```

Questions

- Q3 (continued): Explain why the mask is needed.

The mask is needed to preserve the autoregressive property of the language model. When predicting the next token at a given position, the model must only use the current token and the previous tokens, and must not access any future tokens. Without a mask, each position could attend to the whole sequence, including future characters, which would leak information from the target and make the prediction task meaningless. Therefore, the causal mask ensures that each token can only attend to itself and earlier positions.

For example, if the block size is 4, the attention score matrix has shape 4×4 . Before masking, each position could in principle attend to all positions:

$$\begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix}$$

After applying the causal mask, only the lower-triangular part is kept:

$$\begin{bmatrix} * & 0 & 0 & 0 \\ * & * & 0 & 0 \\ * & * & * & 0 \\ * & * & * & * \end{bmatrix}$$

This means that:

- position 1 can only attend to position 1,
- position 2 can attend to positions 1 and 2,
- position 3 can attend to positions 1, 2, 3,
- position 4 can attend to positions 1, 2, 3, 4.

Hence, when predicting the token at a given position, the model cannot look ahead to any future token.

4. Multi-head Attention

- Q4 (10% marks): Implement the missing parts in the multihead attention class.

```

class MultiHeadAttention(nn.Module):
    """
    Multiple self-attention heads running in parallel.

    Each head independently computes attention over the same input.
    Their outputs are concatenated and linearly projected back to the
    embedding dimension (n_embd).
    """

    def __init__(self, num_heads: int, head_size: int):
        super().__init__()

```

```

# ----- #
# -----[YOUR CODE HERE]----- #
# ----- #

self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)]) #head_size determines the capacity c
self.proj = nn.Linear(num_heads * head_size, n_embd) #Combine all the heads.

self.dropout = nn.Dropout(dropout)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Args:
        x: (B, T, n_embd)

    Returns:
        out: (B, T, n_embd)
    """
    # ----- #
    # -----[YOUR CODE HERE]----- #
    # ----- #

    out = torch.cat([h(x) for h in self.heads], dim=-1) #Sending all heads' output into one vector.
    out = self.proj(out)
    out = self.dropout(out)

    return out

```

5. MLP stage

- Q5 (7.5% marks): Implement the missing parts in the FeedForward attention class.

```

class FeedForward(nn.Module):
    """
    Position-wise Feed-Forward Network (single-hidden layer MLP)
    This module is applied independently to each position in the sequence.

    Input: (B, T, n_embd)
    Output: (B, T, n_embd)
    """

    def __init__(self, n_embd: int):
        super().__init__()

        # The Transformer FFN consists of two linear layers with a non-linearity in between.
        # The inner layer expands the dimensionality by a factor of 4,
        # then projects back to the original embedding size.

        # ----- #
        # -----[YOUR CODE HERE]----- #
        # ----- #

        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd), #Two linear layer
            nn.ReLU(), #Non-linear layer between
            nn.Linear(4 * n_embd, n_embd), #Projects back
            nn.Dropout(dropout),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Applies the feed-forward network independently at each time step.
        No interaction occurs across sequence positions.
        """
        return self.net(x)

```

Questions

- Q6 (10% marks): Explain the relationship between the hyperparameters `n_head` and `head_size` in the `MultiHeadAttention` class. Consider their role within the decoder block (i.e., attention + feed-forward).

Answer to Q(6) In `MultiHeadAttention`, `n_head` is the number of attention heads and `head_size` is the output dimension of each head. They are usually chosen so that

$$n_{\text{head}} \times \text{head_size} = n_{\text{embd}},$$

so that after concatenating all head outputs, the result returns to the full embedding dimension.

For example, if `n_embd = 64` and `n_head = 4`, then we usually set `head_size = 16`. Each head produces a representation of size 16, and after concatenation the output size becomes $4 \times 16 = 64$.

Their role in the decoder block is that they determine how the attention sublayer allocates its capacity. A larger `n_head` allows the model to learn more different attention patterns in parallel, while a larger `head_size` gives each head more representational power. The attention sublayer first uses these heads to gather contextual information from previous tokens, and the feed-forward layer then processes each token representation independently. Therefore, these two hyperparameters control how information is extracted in attention before being further refined by the feed-forward network.

6. Decoder block

- Q7 (10% marks): Implement the missing parts in the DecoderBlock class.

```
class DecoderBlock(nn.Module):
    """
    Transformer decoder block: COMMUNICATION followed by COMPUTATION.

    - Communication: masked multi-head self-attention (tokens exchange information)
    - Computation: position-wise feed-forward network (nonlinear transformation per token)

    Your implementation should use the Pre-LayerNorm (Pre-LN) variant:
    LayerNorm is applied before each sublayer, and residual (skip) connections are applied around each sublayer.
    """

    def __init__(self, n_embd: int, n_head: int):
        # n_embd: embedding dimension, n_head: number of attention heads

        super().__init__()

        # ----- #
        # -----[YOUR CODE HERE]----- #
        # ----- #

        head_size = n_embd // n_head #The relation between n_head and head_size we state before.
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (B, T, n_embd) input embeddings

        Returns:
            x: (B, T, n_embd) output after attention + feed-forward, with residuals
        """
        # ----- #
        # -----[YOUR CODE HERE]----- #
        # ----- #
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))

        return x
```

7. GPT Model: Putting it all together

- Q8 (17.5% marks): Implement the missing computation of the loss in the `GPTLanguageModel` class and the `generate` method to sample tokens that autoregressively extend a sequence.

```
class GPTLanguageModel(nn.Module):
    """
    A minimal GPT-style (decoder-only) Transformer language model.

    Components:
    - Token embeddings: map token IDs -> vectors in  $\mathbb{R}^{n_{\text{embd}}}$ 
    - Positional embeddings: encode token position within the context window
    - A stack of decoder blocks: masked self-attention + feed-forward (+ residuals)
```

```

- Final layer norm + linear head: produce logits over the vocabulary

Input:
    idx: (B, T) integer token IDs

Output:
    logits: (B, T, vocab_size)
    loss: scalar cross-entropy loss (if targets are provided)
"""

def __init__(self):
    super().__init__()

    # Token and position embeddings
    self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
    self.position_embedding_table = nn.Embedding(block_size, n_embd)

    # Stack of decoder blocks
    self.blocks = nn.Sequential(
        *[DecoderBlock(n_embd, n_head=n_head) for _ in range(n_layer)]
    )

    # Final normalization and language modeling head
    self.ln_f = nn.LayerNorm(n_embd)
    self.lm_head = nn.Linear(n_embd, vocab_size)

    # Weight initialization (close to the GPT/Transformer convention)
    self.apply(self._init_weights)

def _init_weights(self, module):
    if isinstance(module, nn.Linear):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

def forward(self, idx: torch.Tensor, targets: torch.Tensor | None = None):
    """
    Args:
        idx: (B, T) token IDs
        targets: (B, T) token IDs (optional)

    Returns:
        logits: (B, T, vocab_size)
        loss: scalar loss if targets is provided, else None
    """
    B, T = idx.shape

    # Token embeddings: (B, T, n_embd)
    tok_emb = self.token_embedding_table(idx)

    # Positional embeddings: (T, n_embd)
    # We create positions [0, 1, ..., T-1] on the correct device.
    pos = torch.arange(T, device=idx.device)
    pos_emb = self.position_embedding_table(pos)

    # Add token and positional information (broadcast pos_emb over batch dimension)
    x = tok_emb + pos_emb # (B, T, n_embd)

    # Decoder blocks + final layer norm
    x = self.blocks(x) # (B, T, n_embd)
    x = self.ln_f(x) # (B, T, n_embd)

    # Convert to vocabulary logits
    logits = self.lm_head(x) # (B, T, vocab_size)

    # ----- #
    # -----[YOUR CODE HERE]----- #
    # -----(computation of the cross entropy loss)----- #
    # ----- #

    loss = None
    if targets is not None: #Here, because the example code just has variable xb, the targets set to be
        B, T, C = logits.shape
        logits_flat = logits.view(B * T, C)
        targets_flat = targets.view(B * T)
        loss = F.cross_entropy(logits_flat, targets_flat)

    return logits, loss

@torch.no_grad()

```

```

def generate(self, idx: torch.Tensor, max_new_tokens: int, temperature: float = 1.0):
    """
    Autoregressively sample new tokens.

    Args:
        idx: (B, T) current context (token IDs)
        max_new_tokens: number of tokens to generate
        temperature: > 0, controls randomness (lower is more deterministic)

    Returns:
        idx: (B, T + max_new_tokens) with generated tokens appended
    """
    self.eval()

    for _ in range(max_new_tokens):

        # ----- #
        # -----[YOUR CODE HERE]----- #
        # ----- #

        idx_cond = idx[:, -block_size:]
        logits, _ = self(idx_cond)
        logits = logits[:, -1, :] / temperature
        probs = F.softmax(logits, dim=-1)
        idx_next = torch.multinomial(probs, num_samples=1)
        idx = torch.cat((idx, idx_next), dim=1)

    return idx

```

8. Training specs

```

# Define hyperparameters (global variables). In a larger project, it is better
# practice to store these in a config object or a dataclass to avoid relying on globals.

# ----- #
# Choose the device we want to run on (CUDA > MPS > CPU)
# ----- #
if torch.cuda.is_available():
    device = torch.device("cuda:0")
elif hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

print("Using device:", device)

# ----- #
# Training hyperparameters
# ----- #
batch_size = 16          # B: number of independent sequences (context windows) processed in parallel
block_size = 128         # T: maximum context length (number of tokens used to predict the next token)
max_iters = 5000         # total number of optimization steps
eval_interval = 500      # evaluate the model every eval_interval steps
eval_iters = 200         # number of batches used to estimate train/val loss during evaluation
learning_rate = 3e-4     # a common default for small GPT-style models (with AdamW)

# ----- #
# Model hyperparameters
# ----- #
n_embd = 384             # embedding dimension (must satisfy: n_head * head_size = n_embd)
n_head = 6               # number of attention heads (so head_size = n_embd // n_head = 64 here)
n_layer = 6              # number of decoder blocks
dropout = 0.2            # dropout probability (regularization)

# ----- #
# Build model and optimizer
# ----- #
model = GPTLanguageModel().to(device)

# Print the number of parameters (in millions)
num_params = sum(p.numel() for p in model.parameters())
print(f"Number of model parameters: {num_params/1e6:.2f} million")

# AdamW is the standard optimizer for Transformer language models
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

Using device: cuda:0
Number of model parameters: 10.74 million

```

We now define a function to generate batches of sequences from our training or validation corpus. We will also define a function to obtain estimates of the loss function.

```
# -----
# Mini DataLoader: sample random context windows from the corpus
# -----
def get_batch(split: str):
    """
    Generate a batch of input (x) and target (y) sequences.

    Args:
        split: 'train' or 'val'

    Returns:
        x: (B, T) input token IDs
        y: (B, T) target token IDs (shifted by one position)
    """
    data_split = train_data if split == 'train' else val_data

    # Random starting positions for each sequence in the batch
    ix = torch.randint(len(data_split) - block_size, (batch_size,))

    # Build input and target sequences
    x = torch.stack([data_split[i:i + block_size] for i in ix])
    y = torch.stack([data_split[i + 1:i + block_size + 1] for i in ix])

    # Move to the selected device (CPU / GPU / MPS)
    x, y = x.to(device), y.to(device)

    return x, y

# -----
# Estimate average loss on train and validation sets
# -----
@torch.no_grad()
def estimate_loss():
    """
    Compute an average loss estimate over `eval_iters` batches
    for both training and validation splits.
    """
    out = {}
    model.eval() # disable dropout, use evaluation mode

    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)

        for k in range(eval_iters):
            X, Y = get_batch(split)
            _, loss = model(X, Y)
            losses[k] = loss.item()

        out[split] = losses.mean()

    model.train() # return to training mode
    return out
```

First sanity check: make sure the sampled batches have the correct dimensions.

```
# Verify that the sampled batches have the expected dimensions
xb, yb = get_batch('train')

print("Input batch shape :", xb.shape) # Expected: (batch_size, block_size)
print("Target batch shape:", yb.shape) # Expected: (batch_size, block_size)

Input batch shape : torch.Size([16, 128])
Target batch shape: torch.Size([16, 128])
```

Second sanity check: verify that the data flows correctly through the model.

```
# Verify that the model's forward pass runs without errors
# and produces logits of the expected shape

logits, _ = model(xb)

print("Logits shape:", logits.shape)
# Expected: (batch_size, block_size, vocab_size)
```

```
Logits shape: torch.Size([16, 128, 65])
```

9. Train

```
# Use your CID as the random seed (for reproducibility)
torch.manual_seed(2300963) #Note, my cid is 02300963, but it reports error:leading zeros in decimal integer
#literals are not permitted; use an 0o prefix for octal integers

# -----
# Training loop
# -----
for step in range(max_iters):

    # Periodically estimate loss on both training and validation splits
    if step % eval_interval == 0 or step == max_iters - 1:
        losses = estimate_loss()
        print(f"step {step}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

    # Sample a batch of training data
    xb, yb = get_batch('train')

    # Forward pass: compute logits and loss
    logits, loss = model(xb, yb)

    # Backward pass + parameter update
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

```
step 0: train loss 4.2682, val loss 4.2679
step 500: train loss 2.1167, val loss 2.1681
step 1000: train loss 1.7596, val loss 1.8530
step 1500: train loss 1.6098, val loss 1.7516
step 2000: train loss 1.5234, val loss 1.6660
step 2500: train loss 1.4697, val loss 1.6432
step 3000: train loss 1.4190, val loss 1.5872
step 3500: train loss 1.3824, val loss 1.5814
step 4000: train loss 1.3533, val loss 1.5578
step 4500: train loss 1.3299, val loss 1.5442
step 4999: train loss 1.3088, val loss 1.5406
```

10. Generate text sequences with the trained model

```
# -----
# Generate text using the trained model
# -----

# Start from a minimal context (single token with ID 0)
context = torch.zeros((1, 1), dtype=torch.long, device=device)

# Generate 1500 new tokens autoregressively
generated_tokens = model.generate(context, max_new_tokens=1500)

# Decode token IDs back into characters
print(decode(generated_tokens[0].tolist()))

# -----
# Optionally, write generated text to a file
# -----

#open('output.txt', 'w').write(
#    decode(model.generate(context, max_new_tokens=10000)[0].tolist())
#)
```

```
LARTIUS:
My gracious lord. Well.
```

```
CORIOLANUS:
O more thrive! it condemn, gentlemen's few,
Because more accused to the head winder they
one of his purpose; Let us mim against it see watch himself,
Having some comfort with braving a hope us.
```

```
VOLUMNIA:
O, what is far?
```

```
SICINIUS:
```

Let's the booken? in me; and sit?

ISABELLA:

River's yok. Now what he hast shall queen for he;
And, you were not for the people sir. He meanus no;
How have hell for more kinsman requir'd free pardon.
Even lawful tword Ratcliff; let's here!
A good Greech on, Beckingham, forth, get Mines,
He has will die die more water to those a son
under nothing bonne but one books on the retty of Yorks.
Give the better words.

BENVOLIO:

New so much an all.

AUFIDIUS:

Say, mercy, can.

LEONTES:

Our power!

CORIOLANUS:

We'll come't to straight.

Second Citizen:

Delike your hands, that indeed me are from From thee!

DUKE OF YORK:

'Tis not see.

LUCIO:

All success to give his honour.

DUKE OF YORK:

I will cannot hear a house of him.

SIR STEPHEN SCALUS:

What, is ere the prince?

QUEEN ELIZABETH:

Go, forbear thy underbury.

KING HENRY VI:

Questions

- Q9 (20% marks): Comment on the generated text.
 - Does it learn local syntax?
 - Does it maintain long-term coherence?
 - How does the temperature affect the generated text?
 - What architectural limitation explains its behavior?

Q9. Comment on the generated text.

(1): The generated text suggests that the model has learned **local syntax reasonably well**. It successfully reproduces several surface-level patterns of the training corpus, including speaker names followed by colons, short dialogue exchanges, and sentence-level punctuation. For example, the extract

CORIOLANUS:

We'll come't to straight.

resembles dramatic dialogue in both formatting and local structure. Similarly,

QUEEN ELIZABETH:

Go, forbear thy underbury.

has a Shakespeare-like tone and imperative structure, even though the final word is not meaningful. These examples suggest that the model has captured short-range syntactic and stylistic regularities.

(2): By contrast, the model shows **limited long-term coherence**. While individual lines may appear plausible in isolation, the passage does not maintain a stable topic, argument, or narrative across multiple turns. For instance, the longer extract

*How had safety such of trides upon the worst,
Unconposs! Prosethee-heart, as enter; sir,
If they'er offends with tender to Romeo to him,*

contains fragments that resemble English syntax, but the overall meaning quickly breaks down. This suggests that the model is much better at imitating local continuation patterns than at maintaining global semantic coherence over extended passages.

(3): The effect of temperature can be understood from the equation

$$\text{logits} = \text{logits}[:, -1, :] / \text{temperature}.$$

Here, the logits for the next-token distribution are divided by the temperature before applying softmax. If the **temperature is low** (for example, less than 1), dividing by a small number makes the logits larger in magnitude, so the softmax distribution becomes sharper. As a result, the model places much higher probability on the most likely tokens, making the generated text more deterministic, repetitive, and locally coherent.

If the **temperature is high** (for example, greater than 1), dividing by a larger number reduces the magnitude of the logits, so the softmax distribution becomes flatter. This means lower-probability tokens are sampled more often, increasing diversity and randomness. However, this also makes the generated text more likely to become noisy, less grammatical, and less coherent over longer passages.

(4): The main **architectural limitation** is the model's restricted context window and small capacity. Since it is a minimal decoder-only Transformer with a finite block size, it can only attend to a limited number of previous tokens. As a result, it may learn local patterns such as punctuation, speaker formatting, and short phrase structure, but it struggles to preserve long-range narrative or semantic coherence. In addition, the small model size limits how much structure it can represent, which helps explain why the generated text often looks locally plausible but globally unstable.